Real Estate Knowledge Retrieval System

Master Technical Reference Document

Real Estate Knowledge Retrieval System with Agentic RAG Pipeline

---

Version	2.0.0
Architecture	Agentic RAG (Retrieval-Augmented Generation)
LLM / Embeddings	Google Gemini 2.5 Flash / Gemini Embedding 001
Vector DB	Pinecone Serverless (1536-dim, cosine)
Backend	FastAPI + Uvicorn
Frontend	Streamlit
Deployment	Docker + Docker Compose
CI/CD	GitHub Actions
Observability	SQLite experiment log + structured JSON logger
Domain	Real Estate property knowledge Q&A

1. Project Overview

The Property RAG Assistant is a production-grade question-answering system for real estate domain knowledge. Users submit natural language questions (e.g., 'What are closing costs?') and receive grounded answers drawn exclusively from a curated knowledge base, along with a confidence score indicating how well-supported the answer is.

What distinguishes this system from a plain LLM chatbot is a two-agent validation layer built on top of the standard Retrieve-Augment-Generate pipeline:

- A Critic Agent that validates every generated answer before it reaches the user, scoring how well it is grounded in the retrieved context.
- A Monitor Agent that tracks aggregate quality metrics across all queries to surface systemic knowledge gaps over time.

Key Design Philosophy

The system is built to detect and communicate its own uncertainty. Rather than silently returning a hallucinated answer, the pipeline scores every response and flags low-confidence results — giving users and operators the information they need to trust or question the output.

2. System Architecture

2.1 High-Level Flow

Every user query travels through five sequential stages:

Stage	Component	What Happens
1. Embed	Gemini Embedding 001	User question → 1536-dim L2-normalized vector
2. Retrieve	Pinecone Serverless	Top-3 semantically similar documents returned
3. Generate	Gemini 2.5 Flash	Answer generated using context only
4. Validate	Critic Agent (Gemini)	Confidence score + grounding status computed
5. Monitor	Monitor Agent (in-memory)	Query cycle recorded; patterns tracked

2.2 Component Diagram

The architecture consists of three logical layers stacked on top of each other:

- Retrieval Layer — Pinecone vector database storing 8 real estate knowledge documents as 1536-dimensional embeddings in the 'property_support' namespace.
- Generation Layer — Gemini 2.5 Flash takes the retrieved context and the user question, producing a context-constrained answer.
- Agentic Layer — The Critic Agent and Monitor Agent inspect the generated answer and accumulate quality statistics respectively.

2.3 Technology Stack

Technology	Role
FastAPI	Production-grade async REST API framework (Python)
Uvicorn	ASGI server for running FastAPI
Streamlit	Interactive web UI served on port 8501
Google Gemini 2.5 Flash	Primary LLM for answer generation and critic evaluation

Technology	Role
Google Gemini Embedding 001	Embeds queries to 1536-dimensional vectors
Pinecone Serverless	Managed vector database for semantic retrieval
NumPy	L2 normalisation of embedding vectors before upsert/query
SQLite	Local persistent database for experiment tracking
Python-dotenv	Environment variable loading from .env file
Docker / Docker Compose	Containerised deployment of API + UI services
GitHub Actions	Automated CI/CD pipeline

3. Project Structure

The repository is organized into four main areas: the core application, the agents package, the MLOps package, and supporting infrastructure.

File / Path	Purpose
main.py	FastAPI application, pipeline orchestration, all API endpoints
streamlit_app.py	Streamlit frontend — Ask, Monitor, and Metrics tabs
agents/__init__.py	Package init, exports validate_answer and MonitorAgent
agents/critic_agent.py	LLM-as-judge answer validation, confidence scoring
agents/monitor_agent.py	In-memory query log, aggregate pattern reporting
mlops/logger.py	Structured JSON logger (one JSON line per log event)
mlops/tracker.py	SQLite-backed experiment tracker with feedback support
tests/conftest.py	Pytest fixtures — mocked Pinecone and Gemini clients
tests/test_main.py	Unit + integration tests for all agents and endpoints
Dockerfile	FastAPI service container (Python 3.11-slim)
Dockerfile.streamlit	Streamlit service container
docker-compose.yml	Orchestrates both services with healthcheck dependency
.github/workflows/ci.yml	Lint → Test → Docker build CI pipeline
.github/workflows/cd.yml	Docker build validation on merge to main
.env.example	Template showing required PINECONE_API_KEY and GEMINI_API_KEY
data/experiments.db	SQLite database created at runtime (gitignored)

4. API Endpoints

The FastAPI backend exposes six endpoints. All endpoints return JSON. Interactive documentation is available at /docs (Swagger UI) when the server is running.

4.1 GET /ask

The primary endpoint. Runs the full agentic pipeline and returns an answer along with critic metadata.

Request: GET /ask?q=What+are+closing+costs

Response fields:

- response — The generated answer string
- meta.query_id — Integer ID for the record in SQLite (used for feedback submission)
- meta.confidence_score — Float 0.0–1.0 from Critic Agent
- meta.grounding_status — One of: grounded, partially_grounded, ungrounded
- meta.flagged — Boolean; true when confidence_score < 0.6
- meta.critic_reasoning — One-sentence explanation from the Critic Agent

```
curl "http://localhost:8000/ask?q=What+are+closing+costs"
```

4.2 GET /monitor

Returns the Monitor Agent's aggregate report across all queries processed since server startup (in-memory, resets on restart).

- summary — Total queries, flagged count, flag rate, average confidence
- grounding_distribution — Counts per grounding status
- recent_flagged_queries — Up to 5 most recent flagged query records
- recommendation — Plain-English advisory based on flag rate thresholds

4.3 GET /metrics

Returns persistent performance metrics computed from the SQLite experiment log (survives restarts).

- avg_embed_latency_ms, avg_retrieve_latency_ms, avg_generate_latency_ms, avg_critic_latency_ms — Per-step averages
- avg_total_latency_ms — End-to-end average latency
- p95_total_latency_ms — 95th percentile total latency

- flag_rate, avg_confidence_score — Quality aggregates

4.4 POST /feedback

Submits human thumbs-up or thumbs-down feedback for a completed query. Requires the query_id from the /ask response.

```
curl -X POST http://localhost:8000/feedback \
  -H "Content-Type: application/json" \
  -d '{"query_id": 1, "rating": "up"}'
```

4.5 GET /feedback/summary

Returns aggregate feedback statistics, including agreement_rate (thumbs up where critic did not flag) and disagreement_rate (thumbs down where critic did not flag). These measure how well the Critic Agent's judgment aligns with user satisfaction.

4.6 GET /docs

Swagger UI — auto-generated interactive API documentation available in any browser while the server is running.

5. The Two Agents

5.1 Critic Agent (agents/critic_agent.py)

The Critic Agent implements an LLM-as-judge pattern. After Gemini generates an answer, the same Gemini model is called a second time with a different role: evaluate whether the answer is actually supported by the retrieved context.

What it does

- Receives: the user's question, the retrieved context chunks, the generated answer
- Sends all three to Gemini 2.5 Flash with a special CRITIC_PROMPT system instruction
- Parses the structured JSON response
- Returns a dict with confidence_score, grounding_status, reasoning, and flagged

Grounding status values

Status	Meaning
grounded	Answer directly references facts present in the context
partially_grounded	Answer is mostly supported but includes some inference
ungrounded	Answer cannot be traced back to the retrieved context

Fallback behaviour

If the Gemini response cannot be parsed as JSON (even after stripping markdown code fences), the function returns a safe default: confidence_score=0.5, grounding_status='partially_grounded', flagged=True. This ensures the pipeline never crashes on a malformed critic response.

> **Cost Trade-off**
> The Critic Agent makes an extra Gemini API call for every user query. This adds latency (measured as critic_latency_ms) and token cost. The trade-off is intentional: for a real estate context where incorrect advice could have financial consequences, catching hallucinations is considered worth the overhead.

5.2 Monitor Agent (agents/monitor_agent.py)

The Monitor Agent is a pure Python class with no external API calls. It accumulates a running log of every query processed since the server started and produces aggregate pattern reports on demand.

Internal state

- query_log — List of all query records (question, timestamp, confidence_score, grounding_status, flagged)
- flagged_queries — Subset list of only the flagged records, for fast access

Methods

Method	Purpose
__init__(low_confidence_threshold)	Sets the flag threshold (default 0.6)
record(question, confidence_score, grounding_status, flagged)	Appends a record after every /ask call
get_report()	Computes and returns the full monitoring report dict
_generate_recommendation(flag_rate, avg_confidence)	Returns a plain-English string based on flag rate thresholds

Recommendation logic

- flag_rate == 0.0 — No issues detected
- flag_rate >= 0.4 — High flag rate; expand knowledge corpus
- flag_rate < 0.4 — Moderate flag rate; review flagged queries

Key limitation

The Monitor Agent stores all state in Python process memory. It resets to empty on every server restart. For persistent monitoring history, use the /metrics endpoint, which reads from the SQLite experiment log.

5.3 How the Agents Differ

Aspect	Critic Agent	Monitor Agent
Scope	Per-query (fires on every /ask)	Aggregate (accumulates over time)
External API calls	Yes — one Gemini call per query	No — pure Python arithmetic
Persists across restarts	No	No (use /metrics for persistence)
Primary output	Confidence score + grounding verdict	Pattern report + recommendation
Detects	Single-answer hallucination risk	Systemic knowledge gaps

6. MLOps Layer

Three observability components are built into the system, providing different granularities of visibility into system behaviour.

6.1 Structured JSON Logger (mlops/logger.py)

Every pipeline step emits a structured JSON log line to stdout. Each line contains these fields:

- timestamp — ISO 8601 UTC timestamp
- level — Log level (INFO, ERROR, etc.)
- event — Event name (query_received, embed_complete, generate_complete, etc.)
- query_id — The SQLite row ID (null until the record is written)
- step — Pipeline step name (embed, retrieve, generate, critic)
- latency_ms — Duration of that step
- extra — Additional context dict (e.g., the question text on query_received)

This format is compatible with log aggregation systems such as Datadog, Splunk, or AWS CloudWatch Logs Insights.

6.2 Experiment Tracker (mlops/tracker.py)

The ExperimentTracker class maintains a SQLite database at data/experiments.db. It auto-creates two tables on first run and provides a clean Python API for reading and writing records.

query_log table

Column	Description
id	Auto-increment primary key (returned as query_id in API responses)
timestamp	ISO 8601 UTC timestamp of the request
question / answer	Full text of the question and generated answer
confidence_score	Float from Critic Agent
grounding_status	grounded / partially_grounded / ungrounded
flagged	Integer 0 or 1
embed_latency_ms	Time to embed the query (ms)
retrieve_latency_ms	Time to query Pinecone (ms)
generate_latency_ms	Time for Gemini to generate the answer (ms)
critic_latency_ms	Time for Critic Agent evaluation (ms)
total_latency_ms	End-to-end wall-clock time (ms)

feedback table

Column	Description
id	Auto-increment primary key
query_id	Foreign key to query_log.id
rating	'up' or 'down'
comment	Optional free-text comment
timestamp	When the feedback was submitted

get_metrics() method

Computes aggregate latency statistics across all persisted queries, including p95 total latency calculated by sorting all total_latency_ms values and picking the value at the 95th percentile index. This is the data source for the /metrics endpoint.

6.3 Human Feedback Loop

After receiving an answer, users can submit thumbs-up or thumbs-down feedback from the Streamlit UI. The /feedback/summary endpoint then computes two composite metrics:

- agreement_rate — Proportion of thumbs-up ratings where the Critic did NOT flag the answer. High agreement means the Critic's confidence correlates with user satisfaction.

- disagreement_rate — Proportion of thumbs-down ratings where the Critic did NOT flag the answer. High disagreement reveals cases where the Critic was confident but users were not satisfied — a signal that the critic threshold may need tuning.

7. Knowledge Base

7.1 Current Documents

The knowledge base currently contains 8 hardcoded real estate facts defined in the documents list in main.py. On server startup, all 8 documents are embedded and upserted to Pinecone in the 'property_support' namespace.

Document	Topic
Doc 1	Home inspection coverage — structural integrity, roof, plumbing, electrical, HVAC
Doc 2	Closing costs — 2% to 5% of purchase price; breakdown of fee types
Doc 3	Buyer's agent role — represents purchaser, compensated via seller's commission
Doc 4	Appraisal — determines fair market value; required by lenders
Doc 5	Earnest money — 1% to 3% deposit; applied toward closing costs
Doc 6	Contingency clause — allows withdrawal without penalty if conditions unmet
Doc 7	Title insurance — protects against liens, encumbrances, ownership disputes
Doc 8	Fixed-rate vs adjustable-rate mortgages — how each works

7.2 Embedding and Retrieval

- Model: gemini-embedding-001 with output_dimensionality=1536
- Normalisation: Each embedding vector is L2-normalised using NumPy before upsert
- Index: Pinecone index named 'property-knowledge' using cosine similarity
- Namespace: All documents live in the 'property_support' namespace
- Retrieval: Top-3 nearest neighbours returned per query (top_k=3)

7.3 Adding New Documents

To expand the knowledge base, add new entries to the documents list in main.py following this format:

```
{"id": "9", "text": "Your real estate knowledge here."}
```

The startup event will automatically embed and upsert the new document on next server start. The Monitor Agent will surface if previously flagged queries become grounded after a knowledge gap is filled.

> **Important Limitation**
> Any question outside the 8 hardcoded topics will likely return an ungrounded or low-confidence answer, since the LLM is instructed to answer only from context. This is by design for a demo system, but production use would require a much larger document corpus.

8. Streamlit Frontend

A dark-themed Streamlit application (streamlit_app.py) serves as the user interface, connecting to the FastAPI backend on http://localhost:8000. It has three tabs.

8.1 Ask Tab

- Text input field for entering any real estate question
- Sidebar with 8 sample questions as clickable buttons
- Answer displayed in a styled card
- Confidence score shown as a coloured percentage badge (green >= 75%, yellow >= 50%, red below 50%)
- Grounding status badge: Grounded / Partially Grounded / Ungrounded
- Flagged warning badge displayed when the Critic flags the response
- Critic reasoning shown in a muted italic box below the answer
- Thumbs up / thumbs down feedback buttons that POST to /feedback with the query_id

8.2 Monitor Tab

- Loads the /monitor report when the tab is opened
- Four metric cards: Total Queries, Flagged Queries, Flag Rate, Avg Confidence
- Recommendation displayed in a highlighted box
- Grounding distribution as horizontal progress bars
- List of recent flagged queries with their confidence scores and timestamps
- Refresh button to reload data from the backend

8.3 Metrics Tab

- Loads /metrics and /feedback/summary on tab open
- Latency cards: avg embed, retrieve, generate, critic latency in milliseconds
- P95 total latency highlighted card
- Feedback summary showing agreement and disagreement rates
- Refresh button

8.4 Running the UI Locally

The Streamlit UI requires the FastAPI backend to be running first. Open two terminals:

```
# Terminal 1 — FastAPI backend
uvicorn main:app --reload

# Terminal 2 — Streamlit frontend
streamlit run streamlit_app.py
```

Then open http://localhost:8501 for the UI and http://localhost:8000/docs for the API.

9. Deployment

9.1 Prerequisites

- Python 3.11 or higher
- A Pinecone account with an index named 'property-knowledge' (cosine similarity, 1536 dimensions)
- A Google AI Studio API key with access to Gemini Embedding 001 and Gemini 2.5 Flash
- Docker and Docker Compose (optional, for containerised deployment)

9.2 Environment Variables

Copy .env.example to .env and populate both values. Never commit the .env file — it is in .gitignore.

```
PINECONE_API_KEY=your_pinecone_key_here
GEMINI_API_KEY=your_gemini_api_key_here
```

The CI/CD pipeline reads these from GitHub Secrets (PINECONE_API_KEY and GEMINI_API_KEY) for the integration test job.

9.3 Local Setup

Step	Command
1. Clone repository	git clone https://github.com/Yuvansh1/property-rag-assistant.git
2. Create virtualenv	python -m venv venv && source venv/bin/activate
3. Install deps	pip install -r requirements.txt

Step	Command
4. Configure env	cp .env.example .env # then add your API keys
5. Start API	uvicorn main:app --reload
6. Start UI (optional)	streamlit run streamlit_app.py

9.4 Docker Deployment

Both services can be started together with a single command:

```
docker compose up --build
```

Service	URL
FastAPI (API)	Port 8000 — http://localhost:8000/docs
Streamlit (UI)	Port 8501 — http://localhost:8501

The Streamlit container uses a healthcheck that waits for the FastAPI container to be healthy before starting. This prevents the UI from starting before the API is ready to serve requests.

10. CI/CD Pipeline

10.1 CI Workflow (ci.yml)

Triggers on push or pull request to the main or develop branches. Runs three sequential jobs:

Job	What It Checks
1. Lint	flake8 (style), black (formatting), isort (import order) — all on main.py
2. Test	pytest with coverage across tests/ — requires both API keys as GitHub Secrets
3. Build	Docker build validation (no push) — confirms the image builds successfully

10.2 CD Workflow (cd.yml)

Triggers on merge to main. Currently performs Docker build validation only (no push to a registry). In a production extension, this job would push the built image to Docker Hub, ECR, or GCR and trigger a deployment.

10.3 Test Coverage

The test suite in tests/test_main.py covers:

- normalize() function — correct L2 normalisation behaviour
- Critic Agent — key validation, JSON fallback on bad response, low confidence flagging, grounded detection
- Monitor Agent — record tracking, flag rate calculation, average confidence, report structure, empty state handling, high flag rate recommendation
- /ask endpoint — HTTP status, response shape, meta field presence, missing query parameter validation, response types
- /monitor endpoint — HTTP status, report structure keys, reflection of actual query history

11. Complete Data Flow

This section traces exactly what happens to a single user query end to end.

Stage	Detail
User submits question	GET /ask?q=What+are+closing+costs
Embed query	Gemini Embedding 001 converts question to 1536-dim float vector; NumPy L2-normalises it
Vector search	Pinecone returns top 3 documents from 'property_support' namespace by cosine similarity
Build context	The metadata.text from each match is joined with newlines into a single context string
Generate answer	Gemini 2.5 Flash receives a prompt containing the context and question; instructed to answer from context only
Critic evaluation	Second Gemini call with CRITIC_PROMPT; returns JSON with confidence_score, grounding_status, reasoning, flagged
Monitor record	Monitor Agent appends entry to in-memory query_log and optionally to flagged_queries
SQLite log	ExperimentTracker inserts full record with all latency breakdowns; returns row ID as query_id
JSON logger	Structured JSON event emitted to stdout for each pipeline step
API response	FastAPI returns {response: answer, meta: {query_id, confidence_score, grounding_status, flagged, critic_reasoning}}

Stage	Detail
Streamlit renders	UI displays answer card, confidence badge, grounding badge, critic reasoning; exposes feedback buttons
User submits feedback	POST /feedback with {query_id, rating} writes to SQLite feedback table

12. Known Limitations and Extension Points

12.1 Current Limitations

Knowledge Base Size
Only 8 hardcoded documents. Any question outside these topics will be flagged as ungrounded. Production use requires a significantly larger and more diverse corpus.

Monitor Agent Persistence
The Monitor Agent stores state in Python process memory. All monitoring history is lost on server restart. The /metrics endpoint (backed by SQLite) provides persistence but only at the aggregate level.

Critic Agent Cost
Every /ask request makes two Gemini API calls — one to generate the answer and one for critic evaluation. This doubles the per-request LLM cost and adds latency (~critic_latency_ms).

12.2 Suggested Extensions

- Expand the knowledge base — Load real estate documents (PDFs, HTML) through a proper ingestion pipeline rather than hardcoding strings.
- Persist Monitor Agent state — Write monitor records to the SQLite database so monitoring history survives restarts.
- Production CD — Add image push to a container registry and deployment step to the cd.yml workflow.
- Re-ranking — Add a cross-encoder re-ranker between retrieval and generation to improve context quality.
- Streaming responses — Use Gemini's streaming API to return tokens progressively for better perceived latency.
- Authentication — Add API key or OAuth middleware to the FastAPI app for production access control.
- Multi-namespace support — Separate Pinecone namespaces per property type or region for domain-specific retrieval.

13. Quick Reference

13.1 API Endpoints

Endpoint	Purpose	Returns
GET /ask?q={question}	Full agentic pipeline	Answer + confidence + grounding
GET /monitor	Monitor Agent report	Aggregate pattern stats + recommendation
GET /metrics	SQLite-backed metrics	Per-step latency + p95 + quality rates
POST /feedback	Submit user rating	Thumbs up/down for a given query_id
GET /feedback/summary	Feedback aggregates	Agreement and disagreement rates
GET /docs	Swagger UI	Interactive API documentation

13.2 Key Configuration Values

Parameter	Value
Pinecone index name	property-knowledge
Pinecone namespace	property_support
Embedding model	gemini-embedding-001
Embedding dimensions	1536
Retrieval top-k	3 documents per query
LLM model	gemini-2.5-flash
Critic flag threshold	confidence_score < 0.6
Monitor low confidence threshold	0.6 (constructor parameter)
SQLite database path	data/experiments.db
FastAPI port	8000
Streamlit port	8501

13.3 Dependency Reference

Package	Purpose
fastapi	REST API framework
uvicorn	ASGI server
pinecone	Vector database client
google-genai	Gemini LLM and embedding client
numpy	Vector normalisation
python-dotenv	Environment variable loading
streamlit	Web frontend
requests	HTTP client used in Streamlit to call the backend API